Product Requirements Document

WorkoutGen — Intelligent Workout Generator

IB Computer Science Internal Assessment

Client: Avery Swift, Personal Trainer

February 2026

1. Project Overview

WorkoutGen is a web-based application that intelligently generates workout routines for gym users. The application solves the problem of repetitive, stale workout routines by generating varied exercise selections while maintaining consistency with essential compound movements. It incorporates a preference-based feedback system inspired by spaced repetition algorithms, allowing users to influence future workout generation based on their exercise preferences.

1.1 Problem Statement

Gym-goers and personal trainers often default to the same exercises session after session, leading to workout staleness, reduced motivation, and physical plateaus. Existing workout apps either generate fully random routines with no historical awareness or lock users into rigid, pre-built programs with no customization. There is a need for an intelligent middle ground that maintains the consistency of foundational compound movements while systematically rotating accessory exercises based on history and user preferences.

1.2 Client Profile

Avery Swift is a personal trainer at Cape Cod Fitness Center in Hyannis, Massachusetts. He designs workout programs for multiple clients and has identified that his current manual approach to workout planning results in repetitive programming, particularly for isolation and accessory movements. He requires a tool that automates exercise rotation while respecting the importance of compound lifts and personal exercise preferences.

1.3 Technology Stack

- **Backend:** Python 3.11+ with FastAPI framework
- **Database:** SQLite 3 (file-based, serverless relational database)
- **Frontend:** TypeScript with React (Vite build tooling)
- **API Communication:** RESTful JSON API
- **Styling:** Tailwind CSS for responsive design

2. Success Criteria

The following success criteria were developed in consultation with the client (see Appendix A1 and A2) and will be used to evaluate the final product:

#	Success Criterion	Priority
1	The system must allow the user to select from at least six muscle groups: Legs, Chest, Back, Shoulders, Arms, and Core.	Must Have
2	The system must generate a workout of five exercises for the selected muscle group, including at least one compound movement that remains consistent across sessions.	Must Have
3	The system must store workout history in a persistent SQLite database so that previously generated workouts can be referenced.	Must Have
4	When generating a new workout, the system must compare against the most recent workout for that muscle group and swap at least two accessory exercises for different ones targeting the same sub-muscle groups.	Must Have
5	The system must allow the user to rate each exercise on a scale of 1 to 5, indicating their preference.	Must Have
6	Exercise ratings must influence future workout generation, with higher-rated exercises appearing more frequently.	Must Have
7	The system must display a workout history view showing past workouts with dates and exercises.	Must Have
8	The system must allow the user to add custom exercises to the exercise database, specifying the muscle group, sub-muscle group, and whether it is a compound or isolation movement.	Must Have
9	The system must validate all user inputs, preventing empty fields, invalid ratings (outside 1–5), or duplicate exercise entries.	Must Have
10	The system must provide a responsive web interface that works on both desktop browsers and mobile devices.	Must Have
11	The system must allow the user to delete or edit existing exercises in the database.	Must Have
12	Each exercise in a generated workout must display its type (compound or isolation).	Should Have

3. System Architecture

3.1 High-Level Architecture

The application follows a client-server architecture with clear separation of concerns:

Frontend (TypeScript/React) → REST API (FastAPI) → SQLite Database

3.2 Backend Architecture

The Python backend is structured using a modular approach:

- **main.py:** Application entry point, FastAPI app initialization, CORS configuration
- **database.py:** SQLite connection management, table creation, seed data loading
- **models.py:** Pydantic models for request/response validation
- **routers/exercises.py:** CRUD endpoints for exercise management
- **routers/workouts.py:** Workout generation and history endpoints
- **routers/ratings.py:** Exercise rating endpoints
- **services/generator.py:** Core workout generation algorithm with history-awareness and preference weighting

3.3 Frontend Architecture

The React frontend is organized by feature:

- **pages/:** Dashboard, Generate Workout, Workout History, Manage Exercises
- **components/:** Reusable UI components (ExerciseCard, RatingStars, WorkoutList, MuscleGroupSelector)
- **services/api.ts:** Centralized API client with typed request/response interfaces
- **types/:** TypeScript interfaces mirroring backend models

4. Database Schema

The SQLite database consists of three main tables with the following structure:

4.1 exercises Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL, UNIQUE
muscle_group	TEXT	NOT NULL
sub_muscle_group	TEXT	NOT NULL
exercise_type	TEXT	NOT NULL (compound/isolation)
equipment	TEXT	NOT NULL
default_rating	INTEGER	DEFAULT 3
user_rating	INTEGER	DEFAULT NULL (1-5)
frequency_weight	REAL	DEFAULT 1.0
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4.2 workouts Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
muscle_group	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4.3 workout_exercises Table

Column	Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
workout_id	INTEGER	FOREIGN KEY -> workouts.id
exercise_id	INTEGER	FOREIGN KEY -> exercises.id
position	INTEGER	NOT NULL (order in workout)
is_compound	BOOLEAN	NOT NULL

5. API Specification

The backend exposes the following RESTful endpoints:

Method	Endpoint	Description	Response
GET	/api/muscle-groups	List all muscle groups	Array of muscle group objects
GET	/api/exercises	List all exercises (filterable by muscle group)	Array of exercise objects
POST	/api/exercises	Add a new exercise	Created exercise object
PUT	/api/exercises/{id}	Update an existing exercise	Updated exercise object
DELETE	/api/exercises/{id}	Delete an exercise	Confirmation message
POST	/api/workouts/generate	Generate a workout for a muscle group	Workout with exercises
GET	/api/workouts/history	Get workout history (filterable by date/muscle group)	Array of past workouts
GET	/api/workouts/{id}	Get a specific workout	Workout detail object
POST	/api/ratings	Rate an exercise (1-5)	Updated rating object
GET	/api/ratings	Get all exercise ratings	Array of rating objects

6. Workout Generation Algorithm

6.1 Overview

The workout generation algorithm is the core computational feature of the application. It produces varied, intelligent workout plans by combining three strategies: compound movement persistence, history-based rotation, and preference-weighted selection.

6.2 Algorithm Steps

Step 1 — Compound Movement Selection: Query all compound exercises for the selected muscle group. Select 1–2 compound exercises. These are persistent: the same compound movements appear in every workout for that muscle group (e.g., Barbell Back Squat always appears in Legs workouts). If the user has rated compound exercises, higher-rated compounds are prioritized.

Step 2 — History Retrieval: Fetch the most recent workout for the selected muscle group from the database. Extract the list of accessory (isolation) exercises used in that workout.

Step 3 — Accessory Exercise Pool: Query all isolation exercises for the muscle group. Remove any that were used in the most recent workout for the same sub-muscle group. This ensures at least two exercises are different from last time.

Step 4 — Preference-Weighted Selection: For the remaining pool, calculate a selection weight for each exercise: weight = base_weight * (user_rating / 3.0). A rating of 5 gives weight 1.67x, a rating of 1 gives 0.33x. Use weighted random selection to fill the remaining 3–4 exercise slots, ensuring sub-muscle group diversity.

Step 5 — Assembly and Storage: Combine compound and accessory selections into a workout of 5 exercises. Store the complete workout in the database with timestamp and exercise positions.

6.3 Pseudocode

```
FUNCTION generate_workout(muscle_group):
    compounds = GET exercises WHERE muscle_group AND type = 'compound'
    selected_compounds = SELECT top 1-2 compounds by rating

    last_workout = GET most recent workout WHERE muscle_group
    last_accessories = GET isolation exercises FROM last_workout

    all_isolations = GET exercises WHERE muscle_group AND type = 'isolation'
    available = all_isolations EXCLUDE last_accessories (by sub_muscle_group)

    IF available.length < needed_count:
        available = all_isolations  // fallback if pool too small

    FOR each remaining slot (5 - compound_count):
        weights = [ex.frequency_weight * (ex.user_rating / 3.0) FOR ex IN available]
        selected = weighted_random_choice(available, weights)
        ADD selected to workout
        REMOVE selected and same sub_muscle exercises from available

    workout = selected_compounds + selected_accessories
    STORE workout in database with timestamp
    RETURN workout
```

7. Exercise Database (Seed Data)

The application ships with the following pre-loaded exercises. Users can add, edit, or delete exercises as needed.

Exercise Name	Muscle Group	Sub-Muscle	Type	Equipment	Default Rating
Barbell Back Squat	Legs	Quadriceps	Compound	Barbell	5
Conventional Deadlift	Legs	Posterior Chain	Compound	Barbell	5
Romanian Deadlift	Legs	Hamstrings	Compound	Barbell	4
Leg Press	Legs	Quadriceps	Compound	Machine	4
Hack Squat	Legs	Quadriceps	Compound	Machine	4
Front Squat	Legs	Quadriceps	Compound	Barbell	4
Bulgarian Split Squat	Legs	Quadriceps	Compound	Dumbbell	3
Seated Hamstring Curl	Legs	Hamstrings	Isolation	Machine	3
Lying Hamstring Curl	Legs	Hamstrings	Isolation	Machine	3
Seated Quad Extension	Legs	Quadriceps	Isolation	Machine	3
Hip Adduction	Legs	Adductors	Isolation	Machine	3
Hip Abduction	Legs	Abductors	Isolation	Machine	3
Standing Calf Raise	Legs	Calves	Isolation	Machine	3
Seated Calf Raise	Legs	Calves	Isolation	Machine	3
Barbell Bench Press	Chest	Pectorals	Compound	Barbell	5
Incline Barbell Bench	Chest	Upper Pecs	Compound	Barbell	4
Dumbbell Bench Press	Chest	Pectorals	Compound	Dumbbell	4
Incline Dumbbell Press	Chest	Upper Pecs	Compound	Dumbbell	4
Cable Fly	Chest	Pectorals	Isolation	Cable	3
Pec Deck	Chest	Pectorals	Isolation	Machine	3
Dumbbell Fly	Chest	Pectorals	Isolation	Dumbbell	3

Push-Up	Chest	Pectorals	Compound	Bodyweight	3
Barbell Row	Back	Lats/Rhomboids	Compound	Barbell	5
Pull-Up	Back	Lats	Compound	Bodyweight	5
Lat Pulldown	Back	Lats	Compound	Cable	4
Seated Cable Row	Back	Mid-Back	Compound	Cable	4
Dumbbell Row	Back	Lats	Compound	Dumbbell	4
T-Bar Row	Back	Mid-Back	Compound	Barbell	4
Face Pull	Back	Rear Delts	Isolation	Cable	3
Straight-Arm Pulldown	Back	Lats	Isolation	Cable	3
Overhead Press	Shoulders	Anterior Delt	Compound	Barbell	5
Dumbbell Shoulder Press	Shoulders	Anterior Delt	Compound	Dumbbell	4
Arnold Press	Shoulders	All Delts	Compound	Dumbbell	4
Lateral Raise	Shoulders	Lateral Delt	Isolation	Dumbbell	3
Front Raise	Shoulders	Anterior Delt	Isolation	Dumbbell	3
Rear Delt Fly	Shoulders	Posterior Delt	Isolation	Dumbbell	3
Cable Lateral Raise	Shoulders	Lateral Delt	Isolation	Cable	3
Barbell Curl	Arms	Biceps	Isolation	Barbell	4
Dumbbell Curl	Arms	Biceps	Isolation	Dumbbell	3
Hammer Curl	Arms	Brachialis	Isolation	Dumbbell	3
Preacher Curl	Arms	Biceps	Isolation	Machine	3
Close-Grip Bench Press	Arms	Triceps	Compound	Barbell	4
Tricep Dip	Arms	Triceps	Compound	Bodyweight	4
Tricep Pushdown	Arms	Triceps	Isolation	Cable	3
Overhead Tricep Extension	Arms	Triceps	Isolation	Dumbbell	3
Skull Crusher	Arms	Triceps	Isolation	Barbell	3
Plank	Core	Rectus Abdominis	Isolation	Bodyweight	4
Hanging Leg Raise	Core	Lower Abs	Isolation	Bodyweight	4

Cable Crunch	Core	Rectus Abdominis	Isolation	Cable	3
Russian Twist	Core	Obliques	Isolation	Bodyweight	3
Ab Wheel Rollout	Core	Rectus Abdominis	Isolation	Ab Wheel	3
Side Plank	Core	Obliques	Isolation	Bodyweight	3
Woodchop	Core	Obliques	Isolation	Cable	3
Dead Bug	Core	Core Stability	Isolation	Bodyweight	3

8. User Interface Specifications

8.1 Pages and Navigation

The application consists of four main views accessible via a navigation bar:

Dashboard: Landing page with muscle group selection buttons (six large, clearly labeled buttons). Shows a summary of recent workout activity.

Generate Workout: Displays the generated workout as a list of exercise cards. Each card shows: exercise name, type (compound/isolation badge), sub-muscle group, equipment needed. Includes a star-rating component (1–5) on each card. A "Generate New" button triggers generation. A "Save Workout" button confirms and stores.

Workout History: Chronological list of past workouts. Each entry shows date, muscle group, and list of exercises. Expandable/collapsible cards for detail. Optional filter by muscle group.

Manage Exercises: Full CRUD interface for the exercise database. Table view with search/filter. Add Exercise form with validation. Edit and Delete actions with confirmation dialog.

8.2 Responsive Design Requirements

- Mobile-first approach: all layouts must work at 375px width minimum
- Desktop layouts may use multi-column grids for exercise cards
- Navigation collapses to a hamburger menu on mobile
- Touch-friendly targets: minimum 44px tap areas for all interactive elements
- Rating stars must be easily tappable on mobile devices

9. Input Validation Rules

Field	Validation Rule	Error Message
Exercise Name	Non-empty, 2–100 characters, unique in database	"Exercise name is required and must be unique."
Muscle Group	Must be one of: Legs, Chest, Back, Shoulders, Arms, Core	"Please select a valid muscle group."
Sub-Muscle Group	Non-empty, 2–50 characters	"Sub-muscle group is required."
Exercise Type	Must be 'compound' or 'isolation'	"Exercise type must be compound or isolation."
Equipment	Non-empty, 2–50 characters	"Equipment field is required."
Rating	Integer between 1 and 5 inclusive	"Rating must be between 1 and 5."

10. Project Structure

Recommended folder structure for VS Code development:

```
workoutgen/
├── backend/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── seed_data.py
│   ├── routers/
│   │   ├── exercises.py
│   │   ├── workouts.py
│   │   └── ratings.py
│   ├── services/
│   │   └── generator.py
│   ├── workoutgen.db
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── main.tsx
│   │   ├── pages/
│   │   │   ├── Dashboard.tsx
│   │   │   ├── GenerateWorkout.tsx
│   │   │   ├── WorkoutHistory.tsx
│   │   │   └── ManageExercises.tsx
│   │   ├── components/
│   │   │   ├── ExerciseCard.tsx
│   │   │   ├── RatingStars.tsx
│   │   │   ├── MuscleGroupSelector.tsx
│   │   │   ├── WorkoutList.tsx
│   │   │   └── NavBar.tsx
│   │   ├── services/
│   │   │   └── api.ts
│   │   └── types/
│   │       └── index.ts
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── tailwind.config.js
└── README.md
```

11. Dependencies

11.1 Backend (requirements.txt)

```
fastapi>=0.104.0
uvicorn>=0.24.0
pydantic>=2.5.0
aiosqlite>=0.19.0  # async SQLite support
```

11.2 Frontend (package.json key dependencies)

```
react: ^18.x
react-dom: ^18.x
react-router-dom: ^6.x
typescript: ^5.x
vite: ^5.x
tailwindcss: ^3.x
axios: ^1.x  # HTTP client for API calls
```

12. Testing Strategy

Each success criterion must have corresponding test cases covering normal, abnormal, and extreme/boundary conditions:

SC#	Test Action	Test Type	Test Data	Expected Outcome
1	Select muscle group	Normal	Click "Legs"	Legs is selected, generation page loads
1	No selection made	Abnormal	Click Generate without selecting	Error: "Please select a muscle group"
2	Generate workout	Normal	Select Legs, generate	5 exercises shown, including a compound
2	Empty exercise DB	Extreme	Delete all Legs exercises	Error: "Not enough exercises"
3	Data persistence	Normal	Generate workout, restart app	Workout appears in history
4	History-based swap	Normal	Generate Legs twice	At least 2 accessories differ
4	Only 3 exercises exist	Extreme	Remove exercises until 3 remain	Generates with available, warns user
5	Rate exercise	Normal	Click 4 stars on Squat	Rating saved, displays 4 stars
5	Invalid rating	Abnormal	API call with rating=0	Error: "Rating must be 1–5"
6	Preference influence	Normal	Rate exercise 5, generate 10 times	High-rated exercise appears more often
7	View history	Normal	Navigate to History	Past workouts displayed with dates
8	Add exercise	Normal	Fill all fields, submit	Exercise added to database
8	Duplicate name	Abnormal	Add existing exercise name	Error: "Exercise already exists"
9	Empty field	Abnormal	Submit with blank name	Error: "Name is required"
10	Mobile view	Normal	Open on 375px viewport	Layout responsive, no overflow
11	Edit exercise	Normal	Change name, save	Name updated in database
11	Delete exercise	Normal	Click delete, confirm	Exercise removed from database
12	Type display	Normal	Generate workout	Each exercise shows compound/isolation badge

13. Development Milestones

Phase	Tasks	Deliverables
Week 1–2	Set up project structure, initialize backend with FastAPI, create SQLite database schema, seed exercise data	Working backend with database, seed data loaded
Week 3–4	Implement exercise CRUD endpoints, build workout generation algorithm, implement history-based rotation logic	API endpoints functional, generation algorithm working
Week 5–6	Build React frontend, implement Dashboard and Generate Workout pages, connect to API	Functional UI with workout generation
Week 7–8	Implement rating system, build Workout History page, build Manage Exercises page	Full feature set complete
Week 9	Input validation (frontend and backend), error handling, responsive design polish	Production-ready application
Week 10	Full testing against all success criteria, bug fixes	Completed test plan with evidence

14. Extensibility Considerations

The application is designed with extensibility in mind, supporting future enhancements without major architectural changes:

- Database migration: SQLite can be replaced with PostgreSQL by changing the connection string; the schema and queries remain compatible.
- Multi-user support: Adding a users table and authentication layer (e.g., JWT tokens) would enable multiple trainers to have separate workout histories and preferences.
- Exercise categories: The sub_muscle_group field supports arbitrary categorization; new muscle groups or sub-groups can be added without schema changes.
- Workout templates: A templates table could store predefined workout structures that the generation algorithm uses as starting points.
- Progress tracking: Adding weight, sets, and reps fields to workout_exercises would enable strength progression tracking.
- API versioning: The /api/ prefix supports versioned endpoints (e.g., /api/v2/) for backward-compatible upgrades.